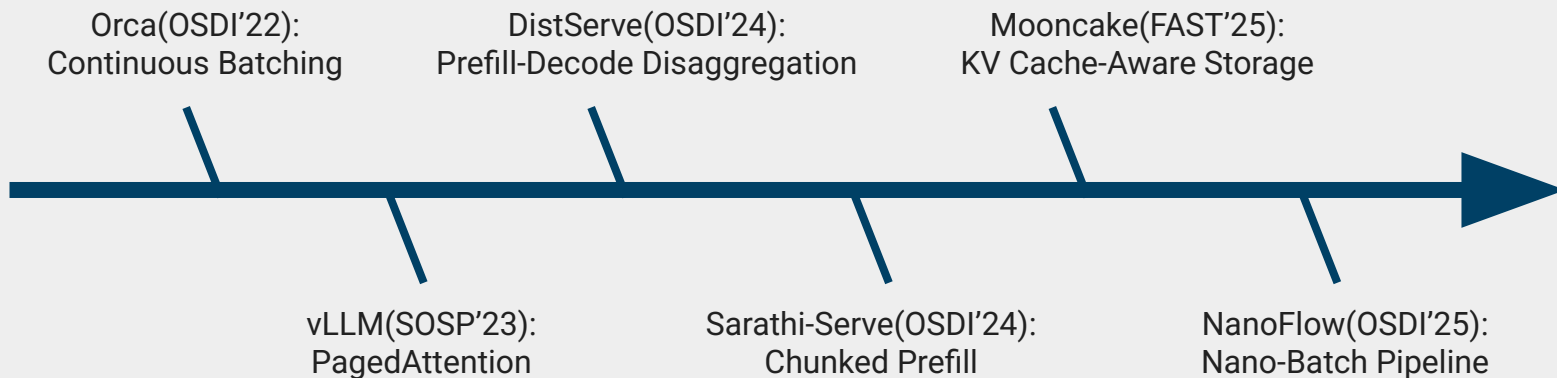


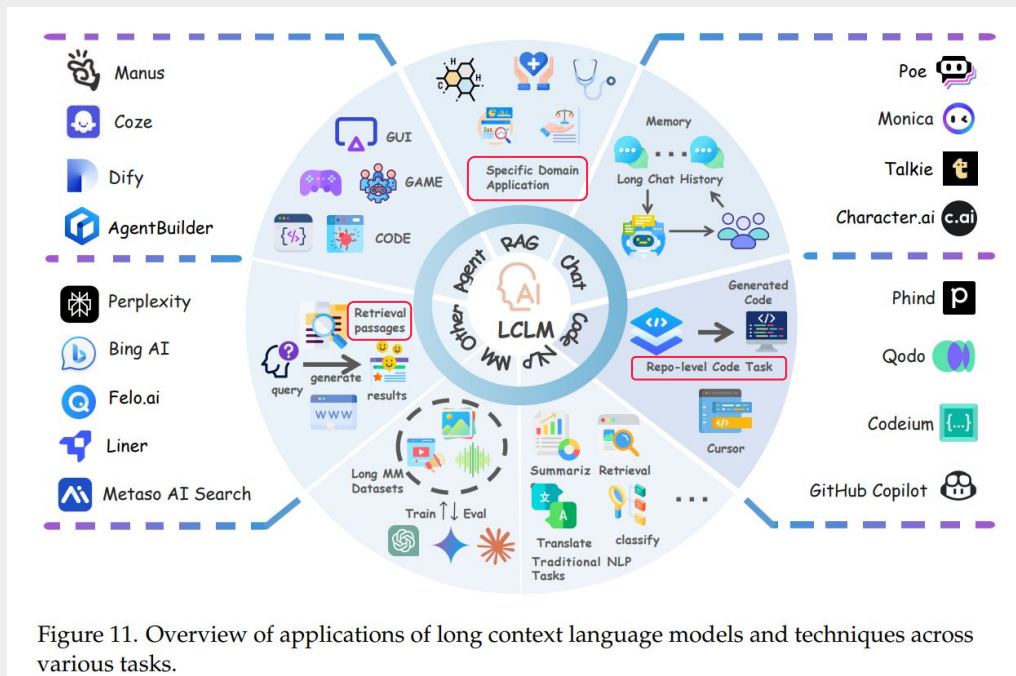
CSE585: Dynamic Disaggregation For Online Serving Systems

Letian Ruan, Fu-Shiang Yang, Terry Shi

Recap

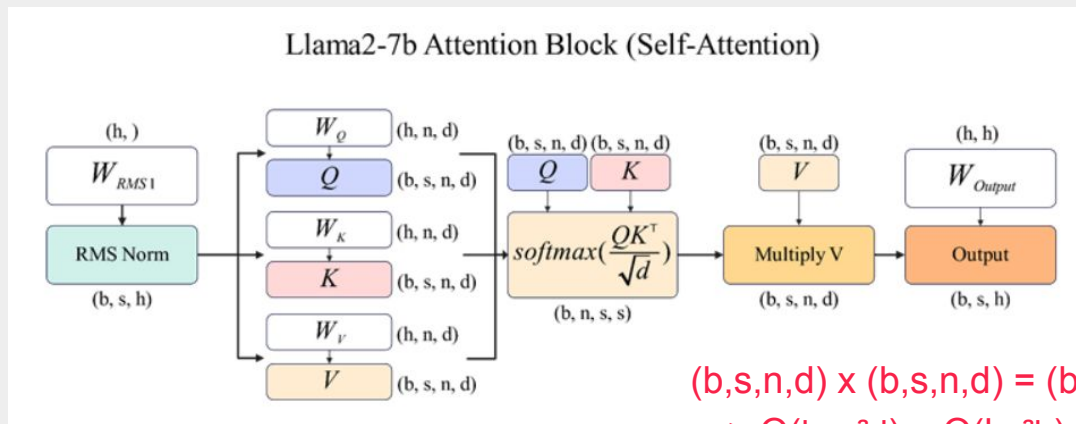


Challenge 1: Long Context



Long Context -> Extremely Large Sequence Length!

Quadratic Computational Complexity $O(bs^2h)$

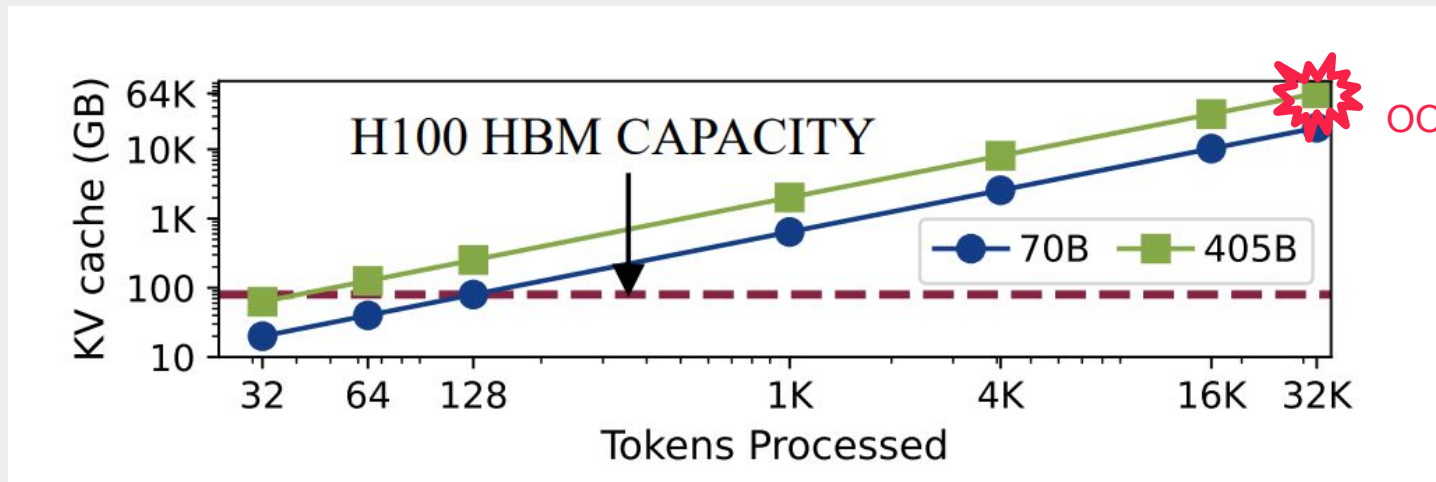


$$(b, s, n, d) \times (b, s, n, d) = (b, n, s, s)$$

$$\rightarrow O(bns^2d) = O(bs^2h)$$

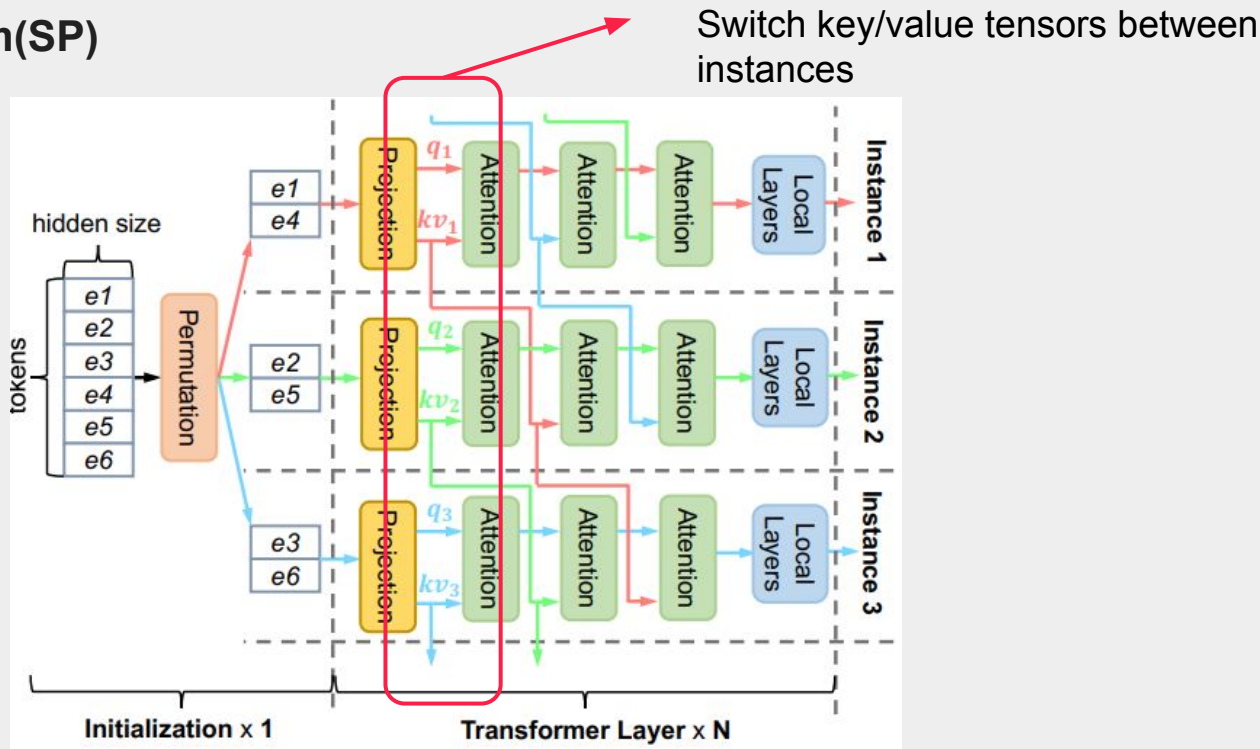
Attention mechanism's compute cost grows **quadratically** with sequence length, causing dramatic **Prefill** overhead when processing long texts and low GPU utilization for later stages

KV Cache Memory Explosion

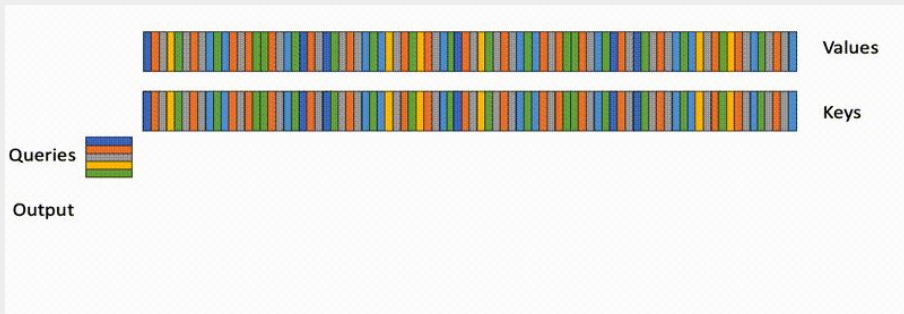


KV Cache size grows **linearly** with sequence length, easily exceeds **Decode** GPU memory limits, becoming the **biggest bottleneck**

Sequence Parallelism(SP)



Split and Reduce Mechanism



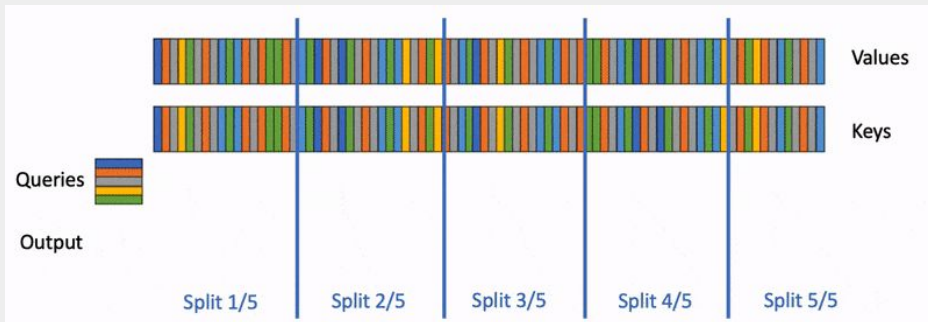
Only separately inside the prefill/decode stage !

—> **How to determine the balanced resource ratio of Prefill:Decode under heterogeneous workload?**

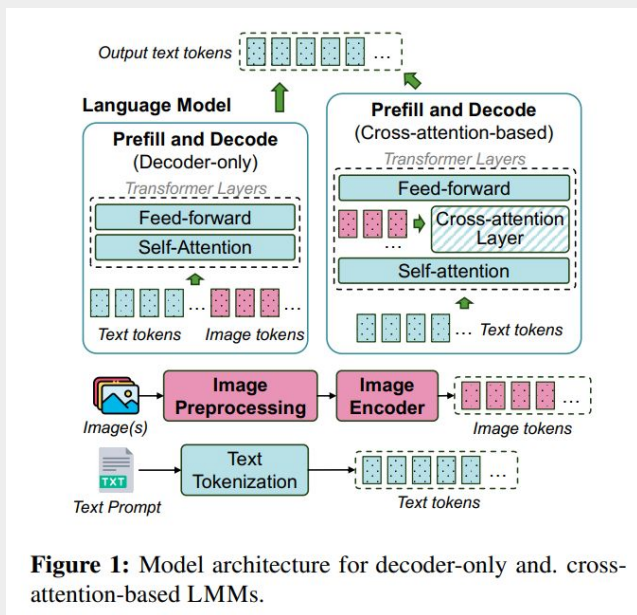


Flash-Decoding works in 3 steps:

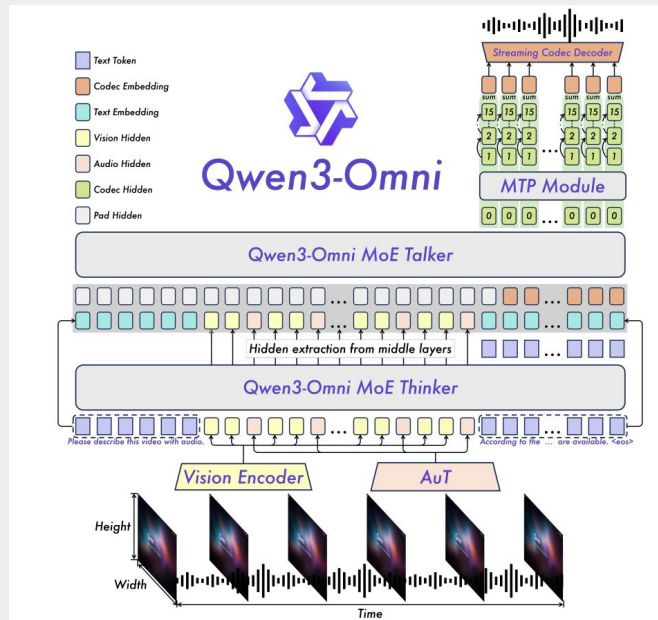
1. **Split** the keys/values of a long sequence to chunks
2. Compute the **partial** attention for all queries
3. **Reduce** over all the splits to get the actual output



Challenge 2: Multimodality

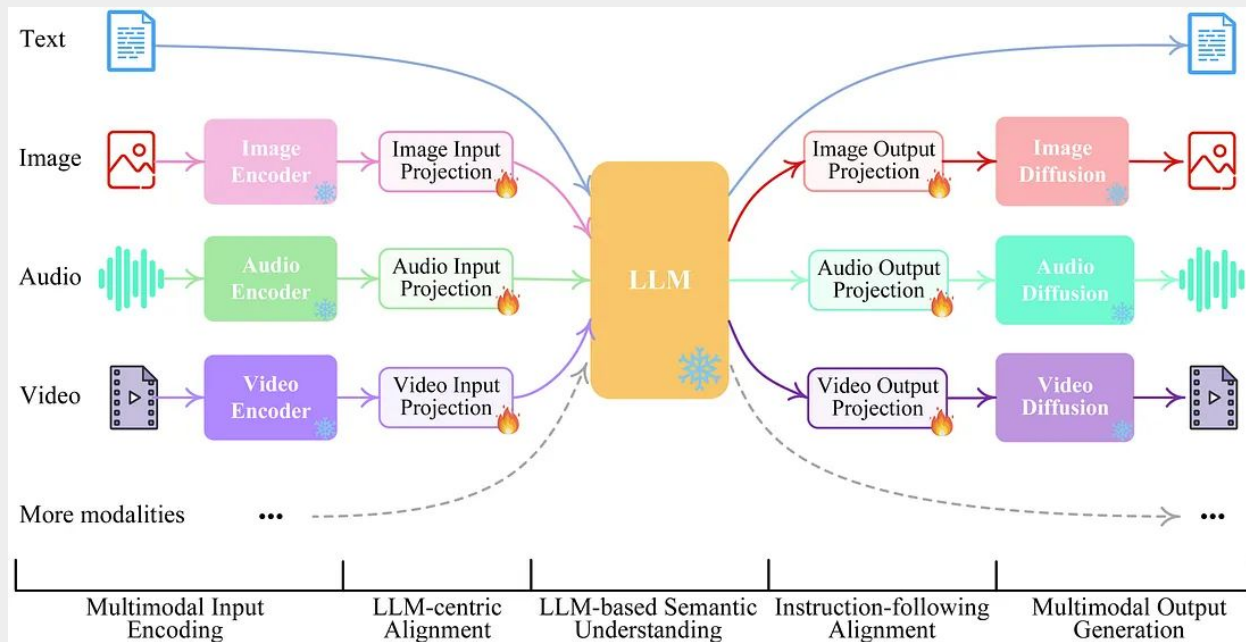


Image+Text to Text



Any to Any

How to handle different multimodal requests?



EPD-Disaggregation

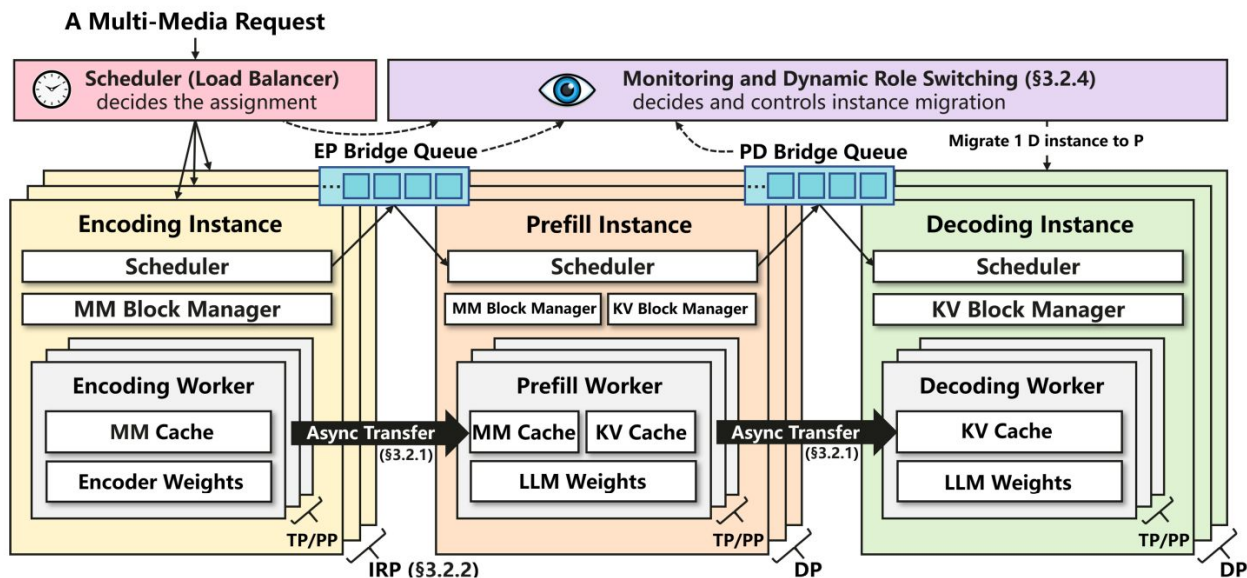


Figure 4: System architecture of the proposed EPD Disaggregated Inference.

What today's papers aim to solve

We already know disaggregation is **necessary** and **achievable** for above situations:

- All-in-one approach can cause degradation in performance or even fail to run at all
- Disaggregation can be achieved by methods mentioned before

BUT, key challenges remain:

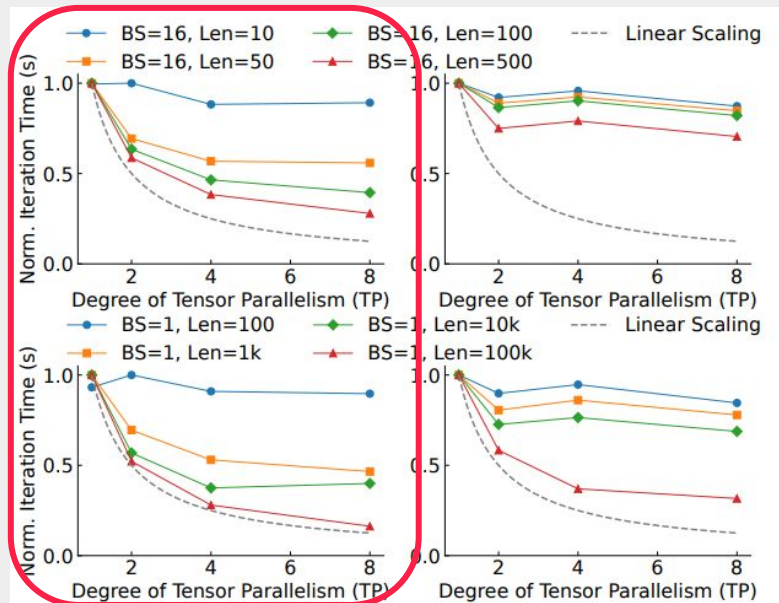
1. How to determine the **optimal Encode/Prefill/Decode ratio**?
2. How to **dynamically adapt** to heterogeneous workloads?
3. How to **coordinate components** for better computation/communication/storage performance?

LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism

SOSP 2024

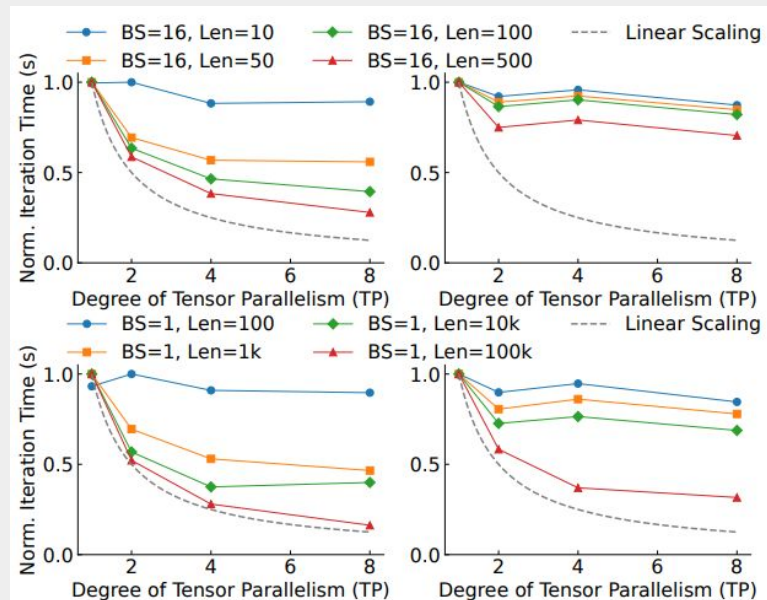
Motivation (The Static vs. Dynamic Mismatch)

1. The resource demand during the prefill phase varies significantly across requests with different input lengths in both computation and GPU memory consumption



Motivation (The Static vs. Dynamic Mismatch)

2. Same request needs different resources in different phases
 - a. Prefill phase(left): Compute-intensive, benefits from high parallelism
 - b. Decoding phase(right): Memory-bound, lightweight computation

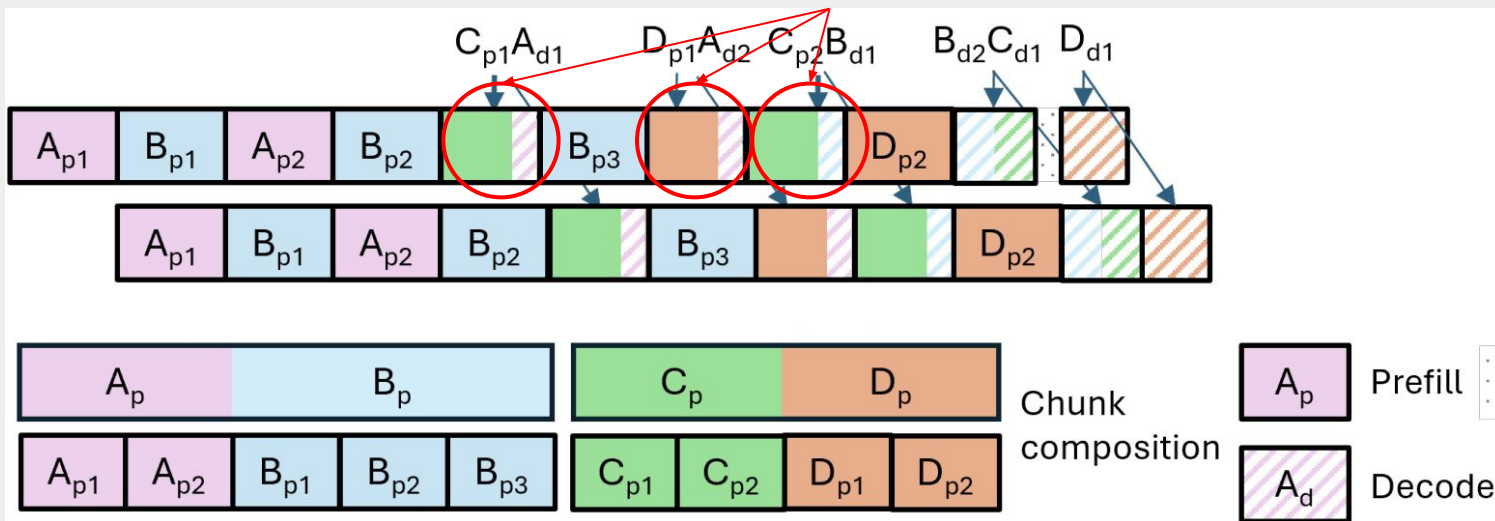


Prior Work

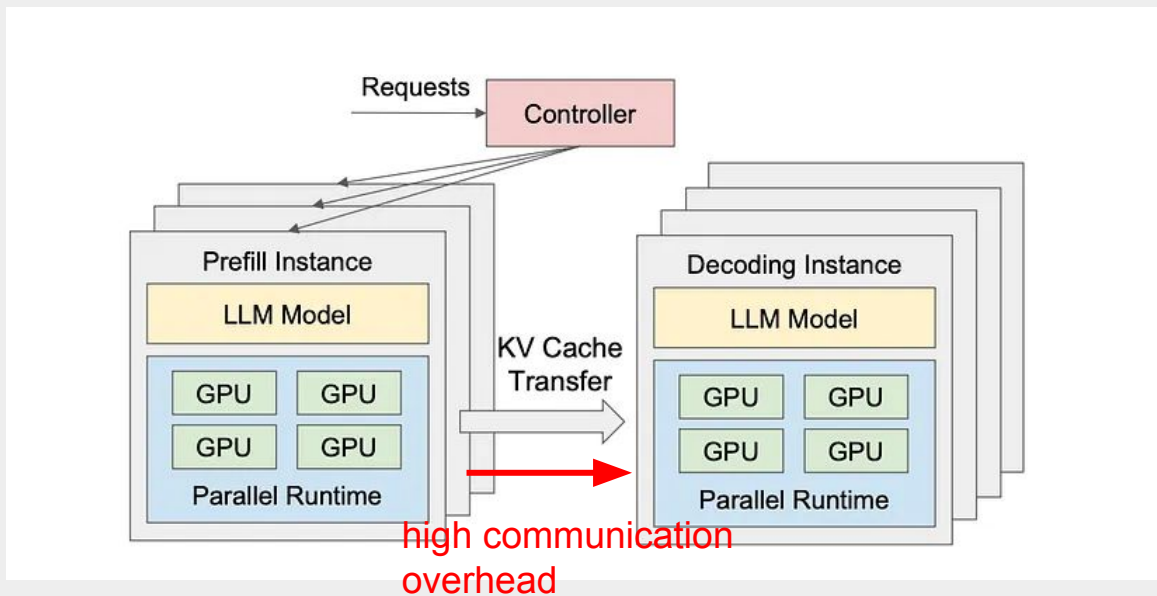
Approach	Issue
Tensor Parallelism (Megatron-LM)	Cannot change without restart
Chunked Prefill	Still has interference
Prefill-Decode Disaggregation	High migration overhead, GPU memory fragmentation

Chunked Prefill

Still Interference

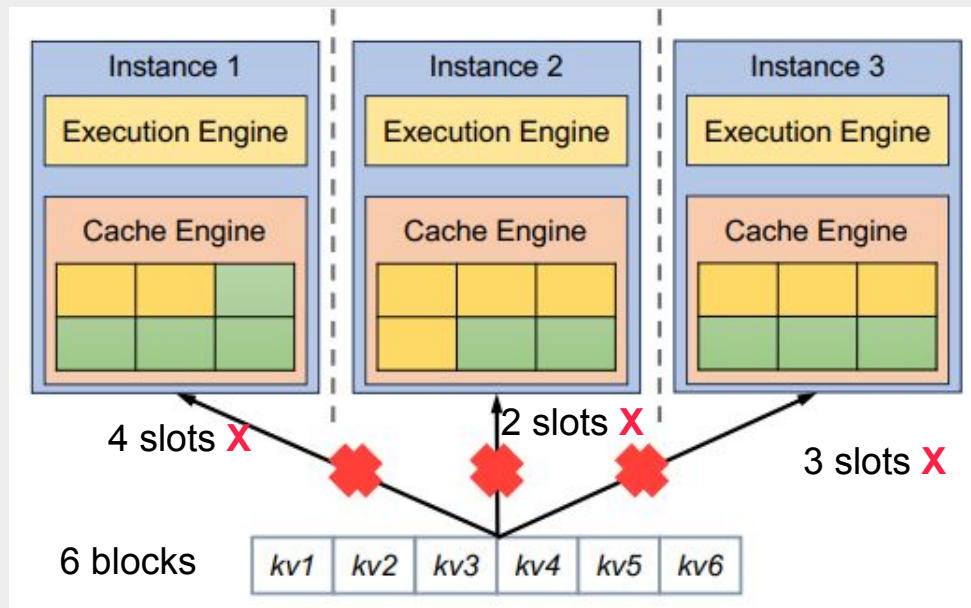


Prefill-Decode Disaggregation



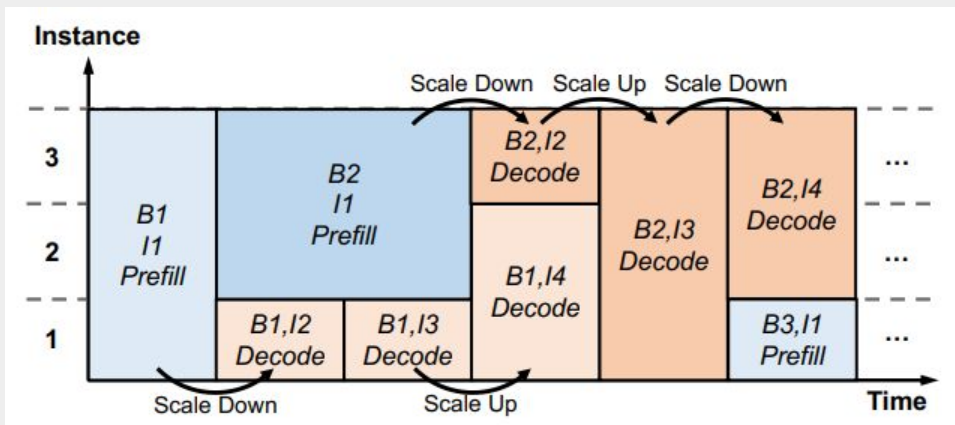
Prefill-Decode Disaggregation

- the entire key-value cache of a request must reside in a single instance causing GPU fragmentation issue.

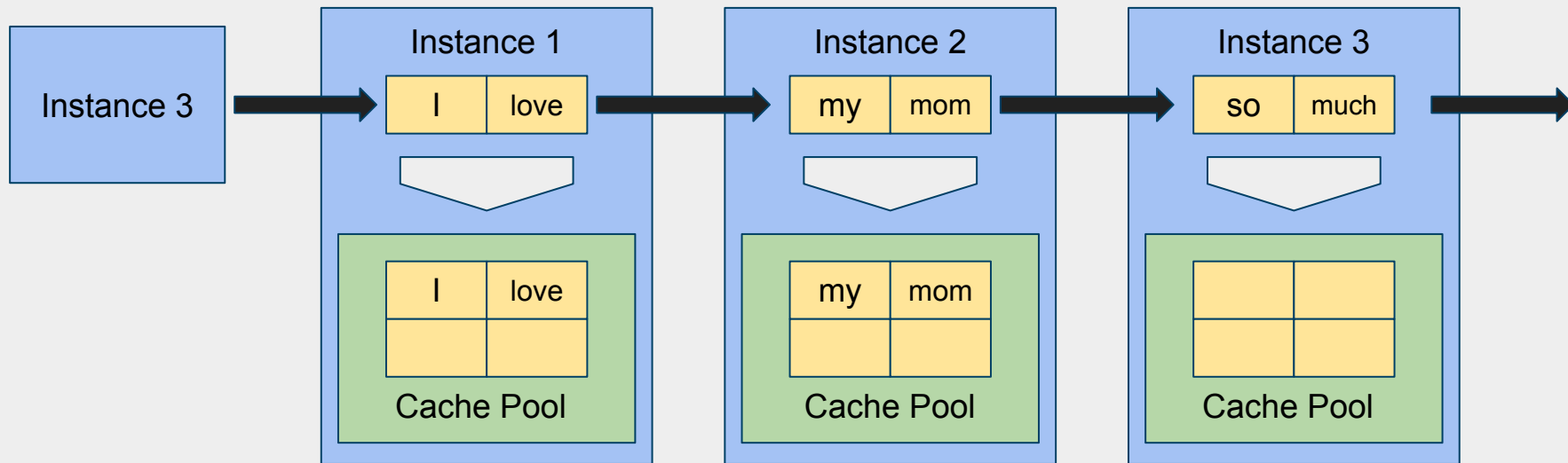


What we want to do - Elastic Sequence Parallelism (ESP)

- Dynamically Adapt Parallelism Per Request & Phase

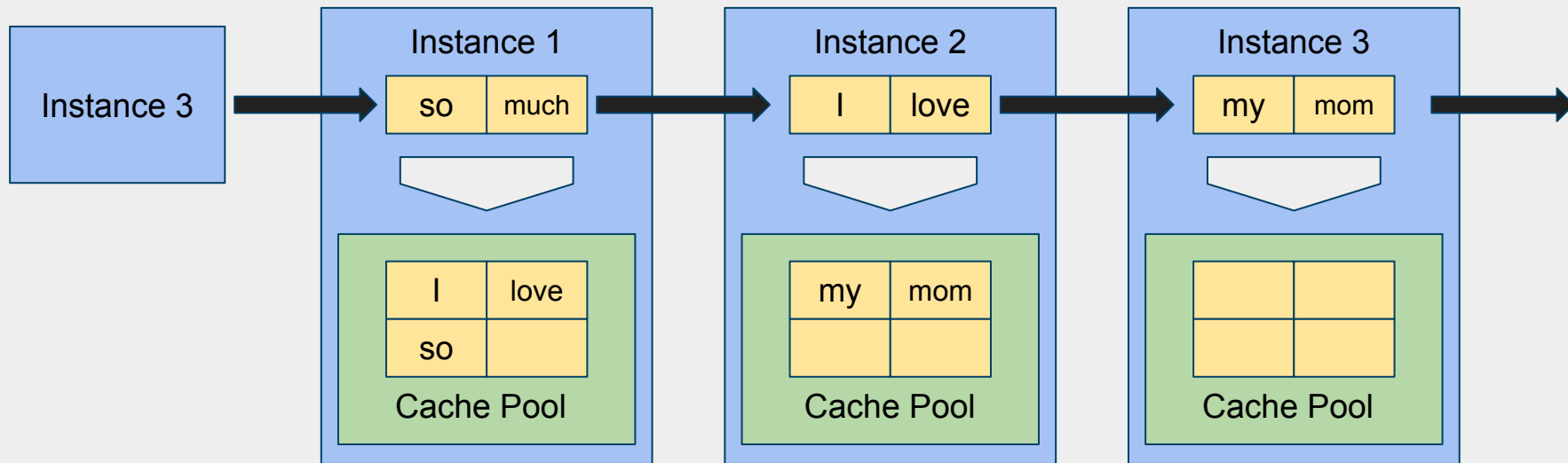


LoongServe Elastic Instances(Scale-down)



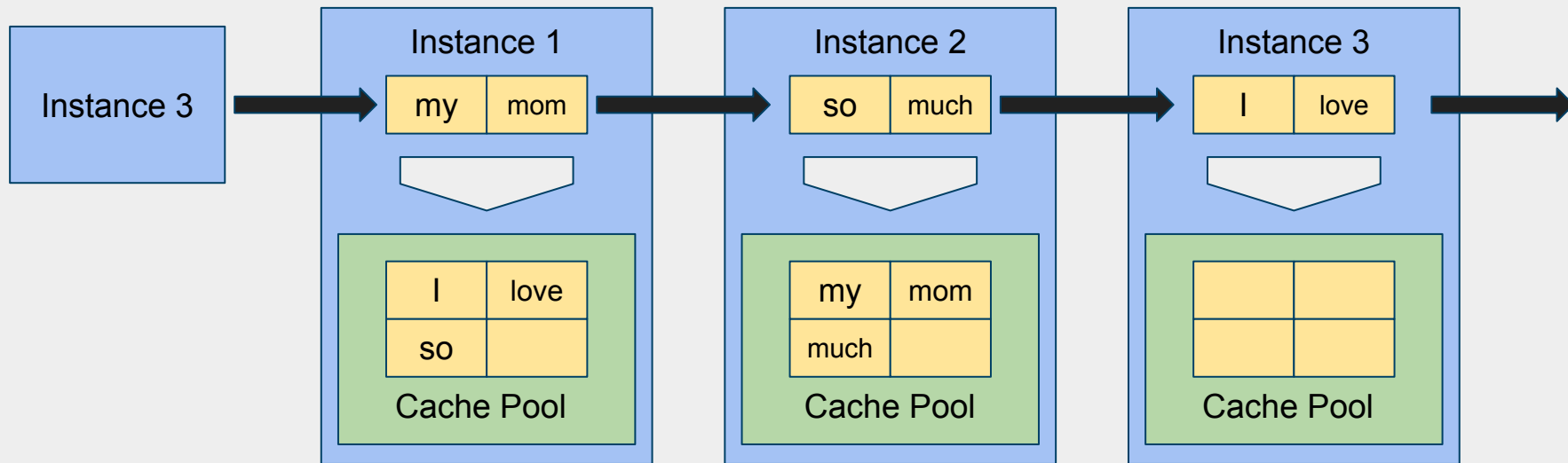
Elastic scale down in the prefill phase.

LoongServe Elastic Instances(Scale-down)



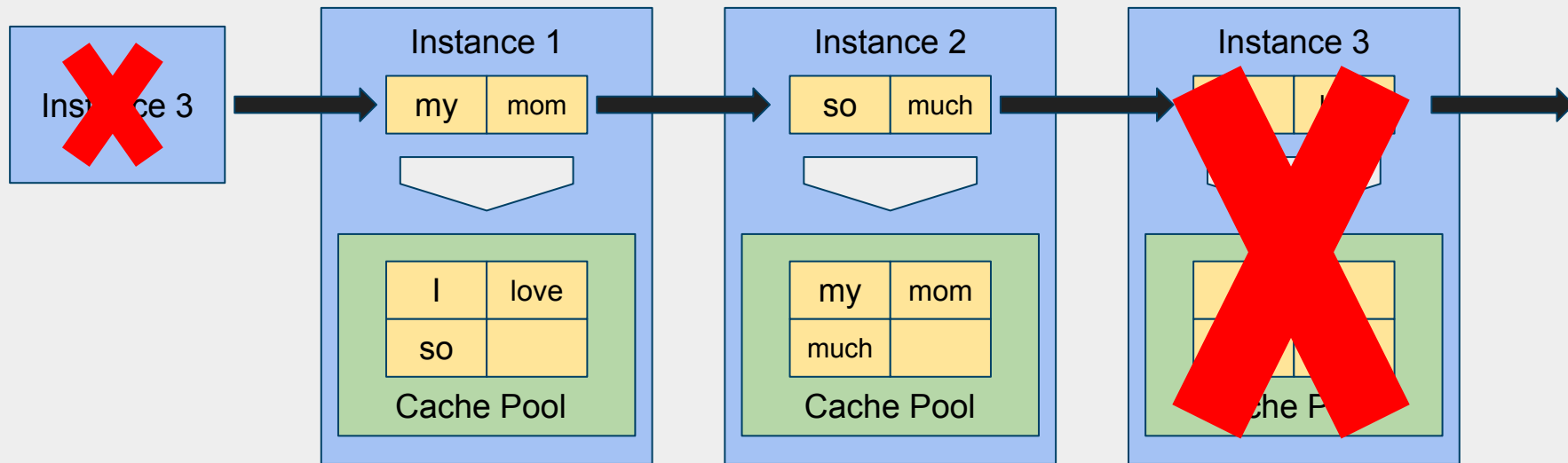
Elastic scale down in the prefill phase.

LoongServe Elastic Instances(Scale-down)



Elastic scale down in the prefill phase.

LoongServe Elastic Instances(Scale-down)

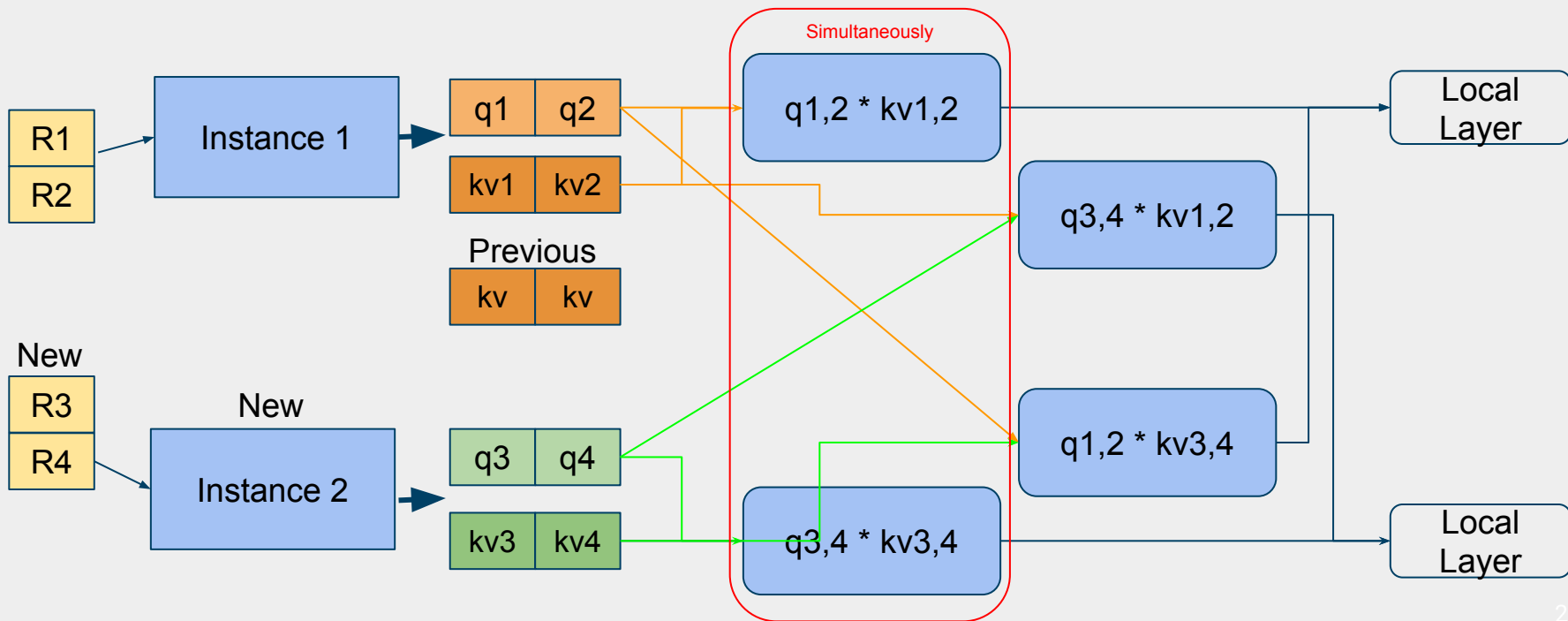


Elastic scale down in the prefill phase.

LoongServe Elastic Instances(Scale-down)

- Reuse prefill communication to place KV cache
- No additional migration overhead

LoongServe Elastic Instances(Scale-up)

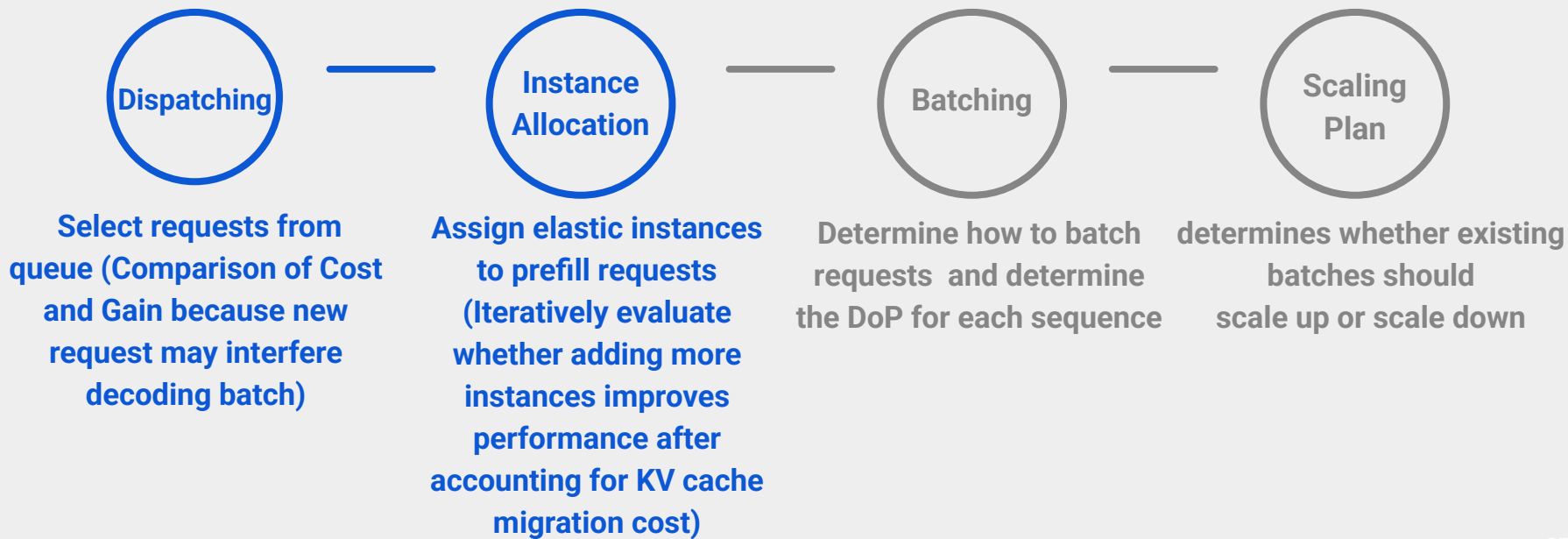


LoongServe Elastic Instances(Scale-up)

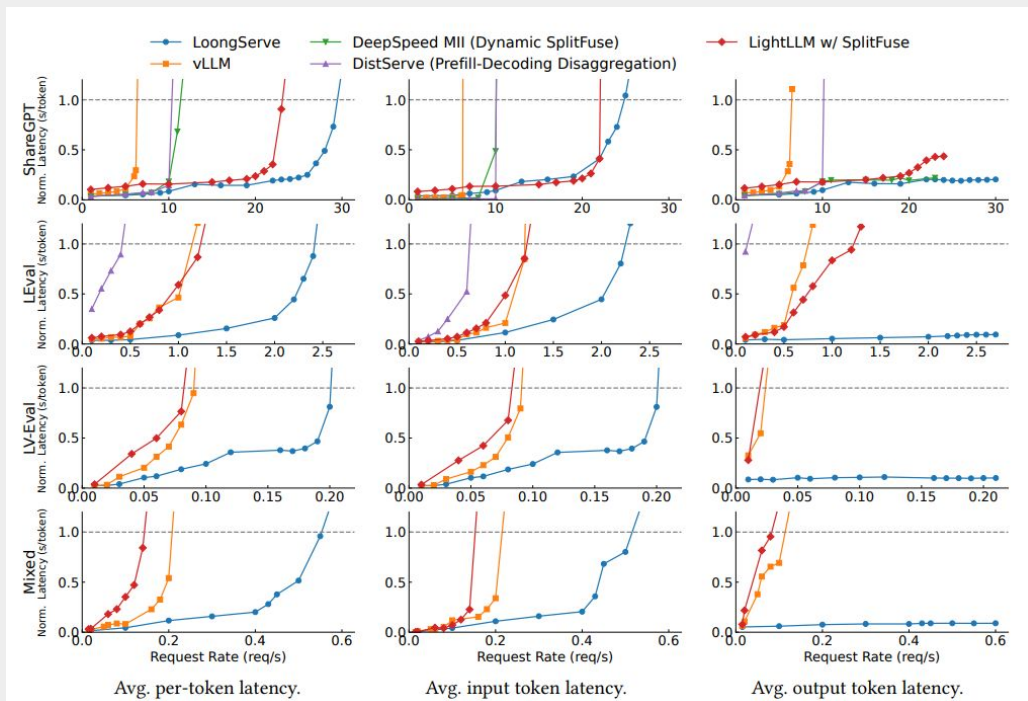
- Add instances without migrating existing KV cache
- Overlap communication with computation
- Token-level cache placement (no locality constraints)



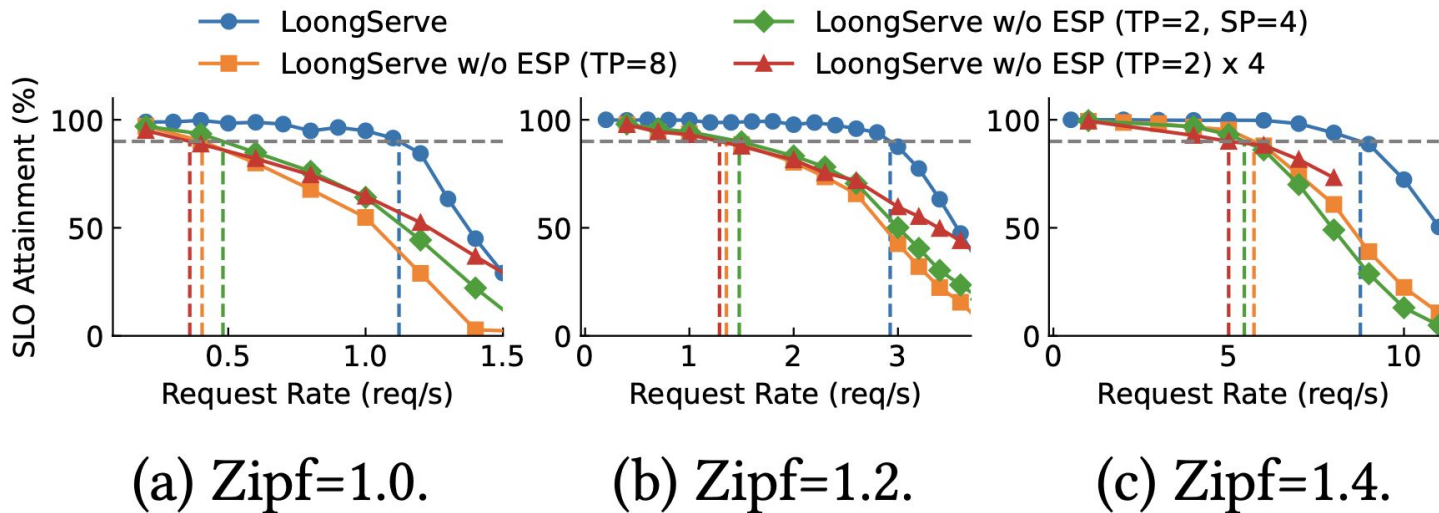
LoongServe Global Manager



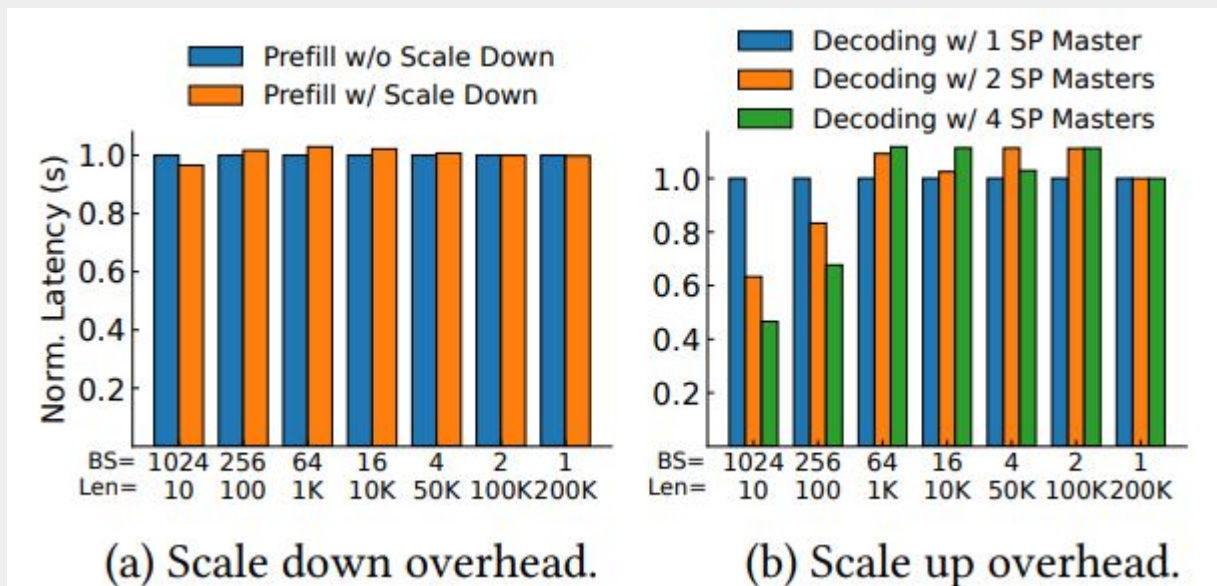
Evaluation : Latency



Evaluation : Elastic Sequence Parallelism



Evaluation : Overhead



Conclusion

- To serve long-context LLMs under dynamic workloads, LoongServe provides a set of elastic scaling mechanisms without additional overhead and a scalable scheduling algorithm for ESP

Limitations

- This method requires sequence parallelism as its foundation. The experiments only compare overhead combined with tensor parallelism, not with other parallelism types.
- LoongServe's ESP only considers attention layers. But FFN layers still have heavy computation, potentially becoming bottlenecks in long context decoding.

Cornserve: Efficiently Serving Any-to-Any Multimodal Models

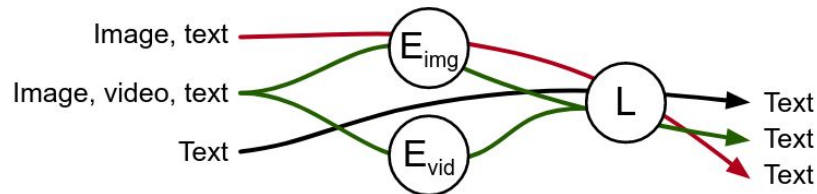
Preprint

Motivation: why do we care?

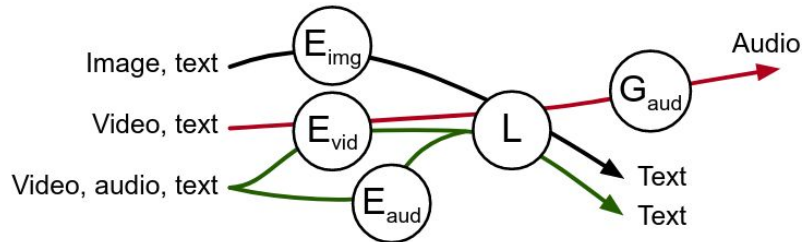
- Workloads are heterogeneous
- Components often have mismatched speeds
- Fixed layouts and fixed plans often waste GPUs or blow up latency
- Goal: **Maximize** throughput, **Minimize GPUs** for target load

Any to Any Models

- Components:
Multimodal encoders, LLM(s),
Multimodal generators
- Requests select subgraphs at runtime
- Example: Qwen2.5-VL, InternVL-3,
Qwen-Image



(a) Multimodal input, text output (MLLM)



(b) Multimodal input, multimodal output

Prior Systems are point solutions

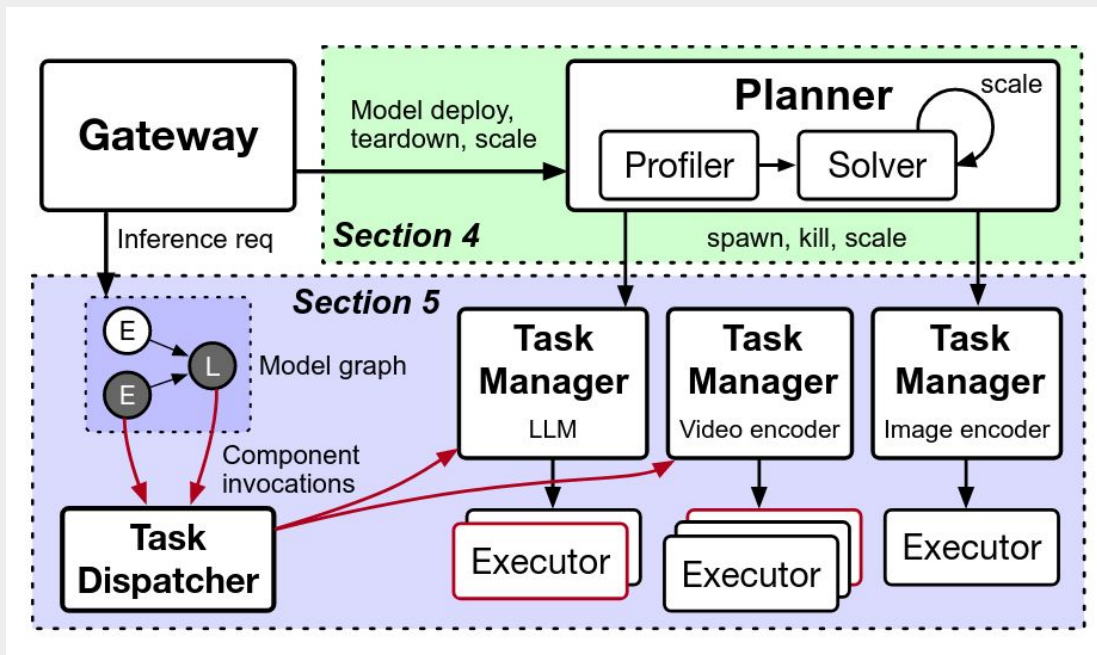
- vLLM (text-only / MLLM input)
- diffusion stacks (image/video generation)
- **No** general planner for Any to Any models

Encoder (E), Prefill (P), Decode (D)

Monolith: put all needed components in a single executor

1920×1080×1 input 100 output 100	4.4	3.2	2.0	2.4	5.5	4.7	3.2	1.9
1920×1080×1 input 100 output 300	3.5	3.2	2.2	2.4	4.0	3.3	2.5	1.9
1920×1080×1 input 1000 output 300	2.8	3.2	2.1	1.8	3.1	2.6	2.2	1.5
1680×1050×1 input 1000 output 300	3.2	3.8	2.5	2.1	3.5	2.9	2.3	1.7
1680×1050×1 input 100 output 300	4.4	4.0	2.6	3.0	4.6	3.9	3.0	2.2
1680×1050×1 input 1000 output 500	2.5	2.6	2.5	2.1	2.7	2.2	1.6	1.6
896×896×2 input 1000 output 300	2.4	2.8	1.5	1.6	3.7	3.0	2.5	1.8
896×896×1 input 1000 output 300	4.2	4.3	2.7	2.6	5.8	4.9	4.0	2.9
	4 Monoliths 2E/3PD 2EP/2D 2E/1P/2D				4 Monoliths 2E/3PD 2EP/2D 2E/1P/2D			

Architecture Overview



Planner

Objective:

1. Max throughput given GPU budget N
2. Min GPUs given target throughput R

Model Computation
Graph

Workload: request
types, sizes, tokens...

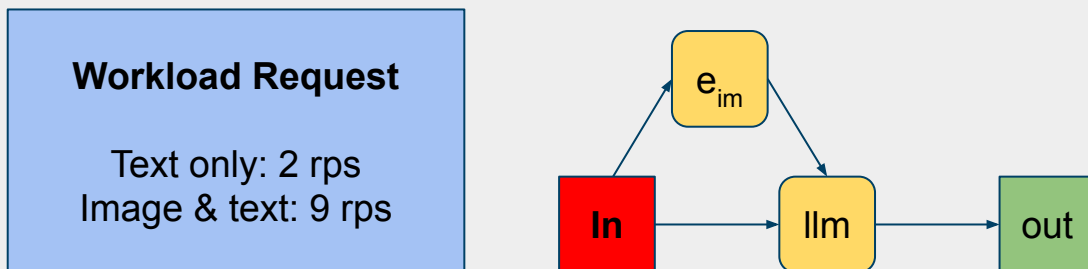
Planner

Profiler

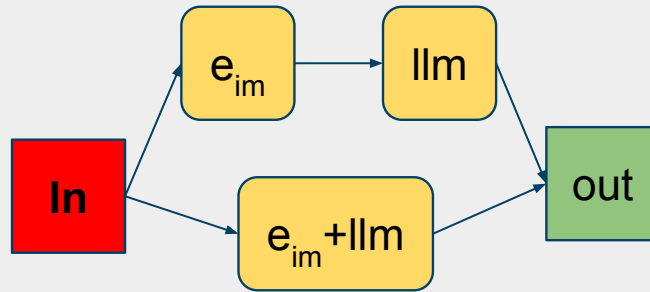
Solver

Deployment Plan:
Executor options +
replica counts + rate
splits

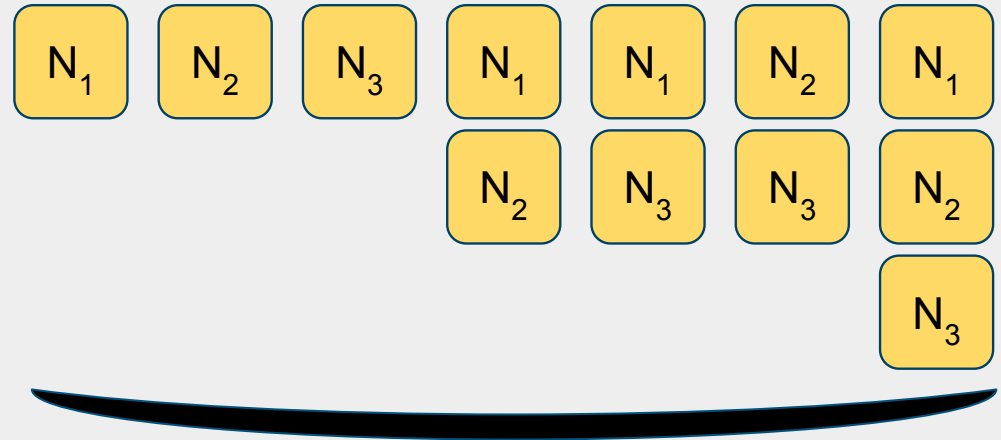
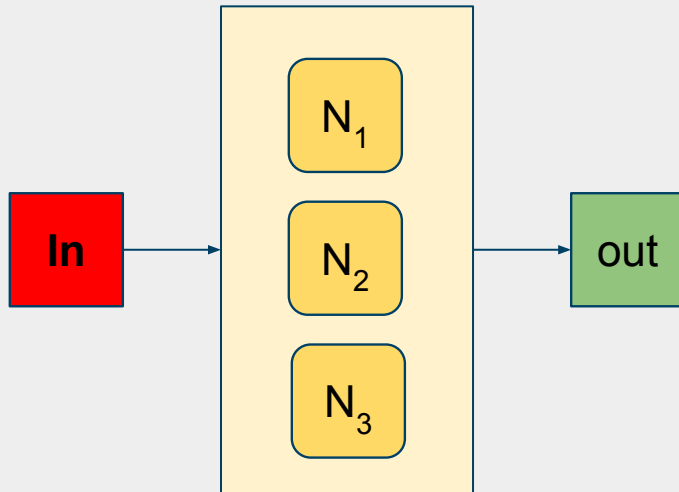
Example Model Workload



Keeping the workspace small

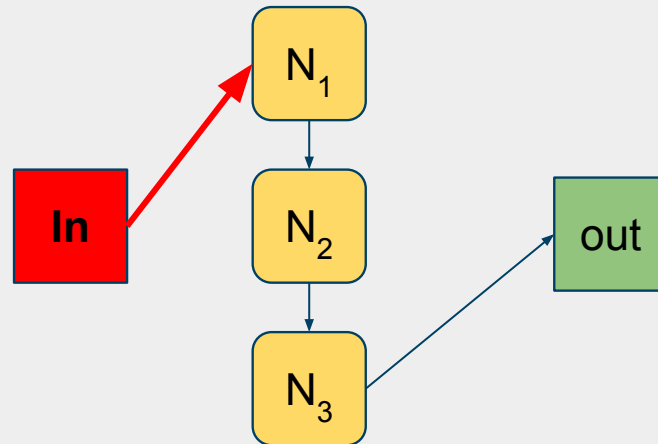


Keeping the workspace small



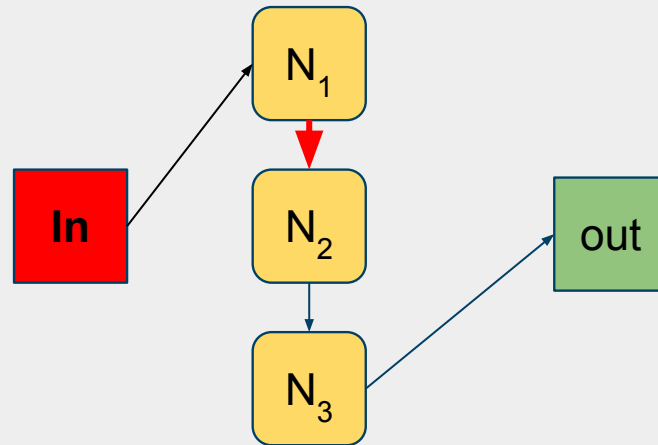
$2^n - 1$ possible
combinations

Keeping the Workspace small



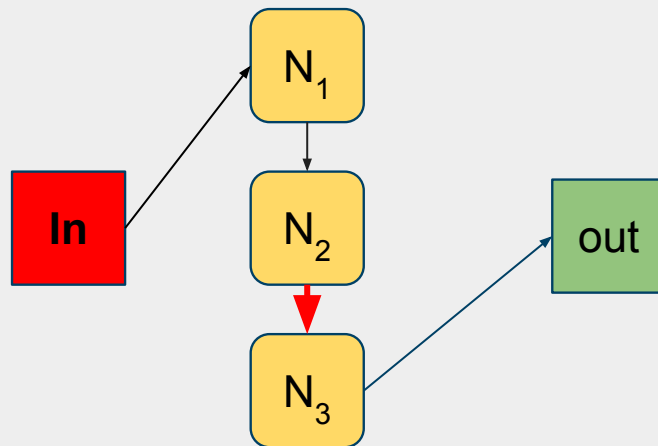
- Naive grouping of n components $\rightarrow 2^n - 1$ configurations
- Reduce to n singletons + 1 monolith = $n + 1$ nodes
- Result: small, expressive search space

Keeping the Workspace small



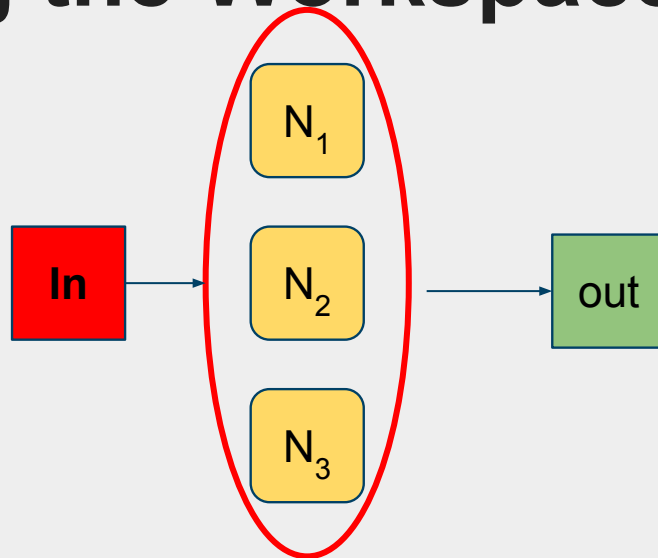
- Naive grouping of n components $\rightarrow 2^n - 1$ configurations
- Reduce to n singletons + 1 monolith = $n + 1$ nodes
- Result: small, expressive search space

Keeping the Workspace small



- Naive grouping of n components $\rightarrow 2^n - 1$ configurations
- Reduce to n singletons + 1 monolith = $n + 1$ nodes
- Result: small, expressive search space

Keeping the Workspace small

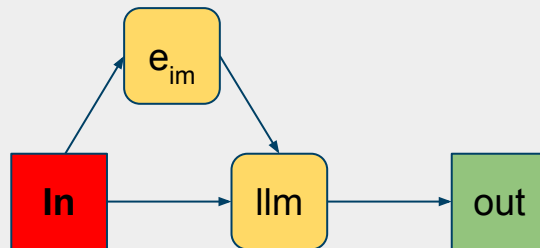


- Naive grouping of n components $\rightarrow 2^n - 1$ configurations
- Reduce to n singletons + 1 monolith = $n + 1$ nodes
- Result: small, expressive search space

Building Executor Graph

Workload Request

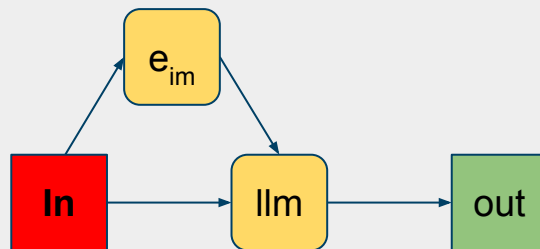
Text only: 2 rps
Image & text: 9 rps



Building Executor Graph

Workload Request

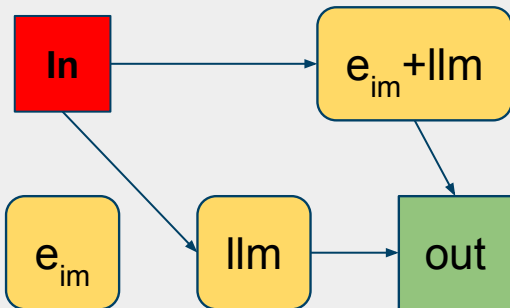
Text only: 2 rps
Image & text: 9 rps



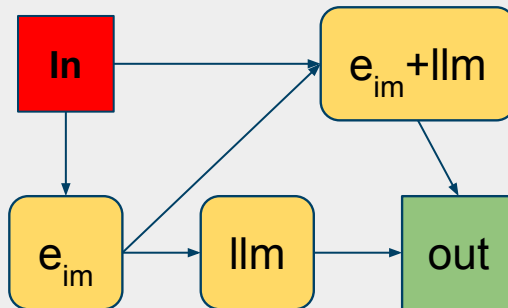
	Img	text	Both	GPUs
e _{im}	10			1
llm		3.33		2
e _{im} +llm		3.03	2.27	2

Building Executor Graph

Text Only inputs



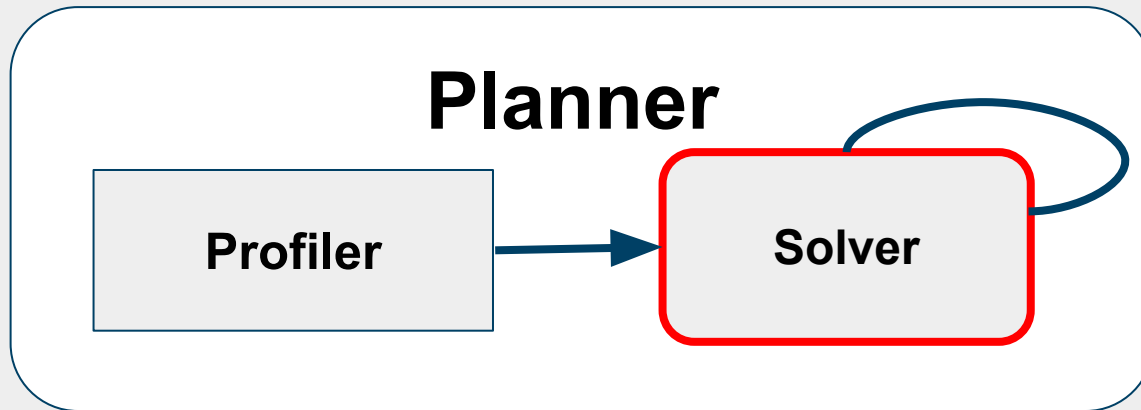
Text + Img inputs



Profiler Results

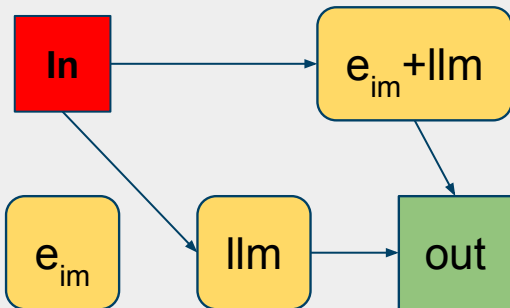
	Img	text	Both	GPUs
e _{im}	10			1
llm		3.33		2
e _{im} + llm		3.03	2.27	2

Solving



Solving: Maximize throughput

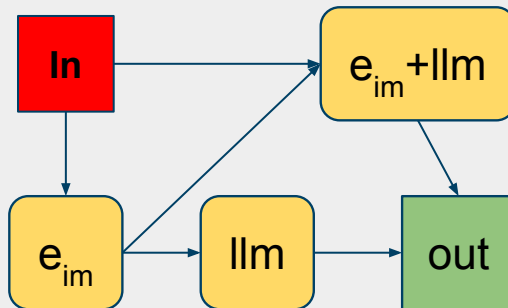
Text Only inputs



What we Know:

#GPUs,
Workload Mix,
Executor Options
Per Replica Speeds -> Profiler

Text + Img inputs



Solver Solves for:

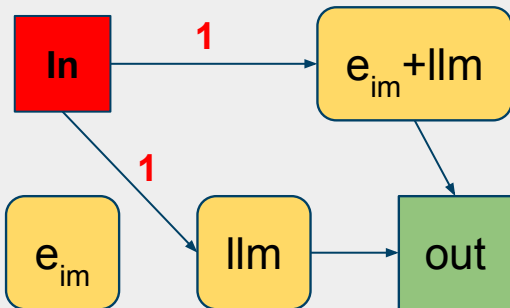
- How to split each request type across valid paths
- How many replicas of each executor so capacity $\geq$ assigned load.

Profiler Results

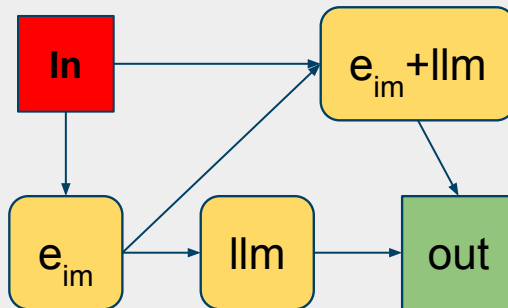
	Img	text	Both	GPUs
e_{im}	10			1
llm		3.33		2
$e_{im} + llm$		3.03	2.27	2

Solving: Maximize throughput

Text Only inputs



Text + Img inputs

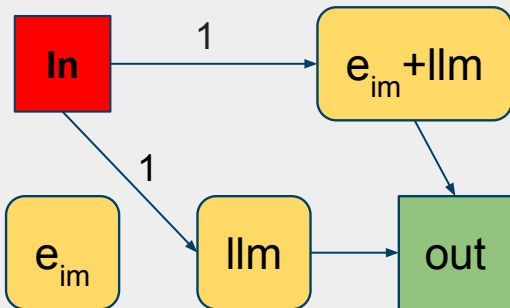


Profiler Results

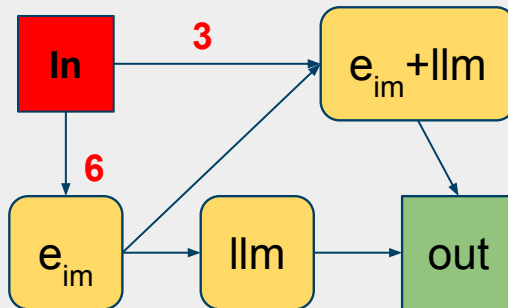
	Img	text	Both	GPUs
e_{im}	10			1
llm		3.33		2
$e_{im} + llm$		3.03	2.27	2

Solving: Maximize throughput

Text Only inputs



Text + Img inputs

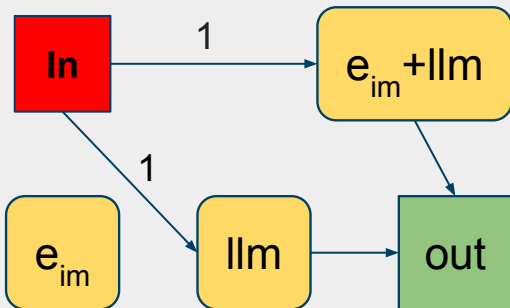


Profiler Results

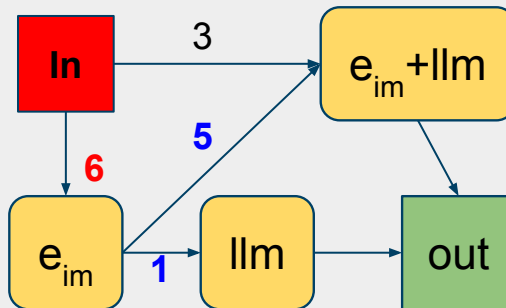
	Img	text	Both	GPUs
e_{im}	10			1
llm		3.33		2
$e_{im} + llm$		3.03	2.27	2

Solving: Maximize throughput

Text Only inputs



Text + Img inputs

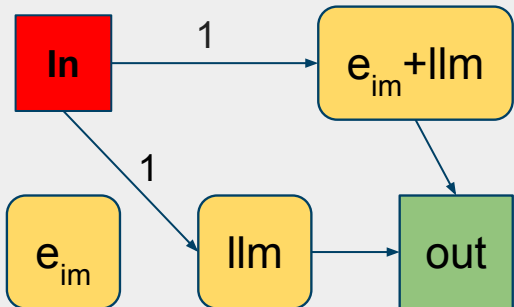


Profiler Results

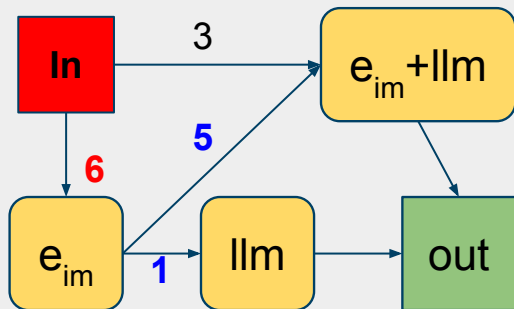
	Img	text	Both	GPUs
e_{im}	10			1
llm		3.33		2
$e_{im} + llm$		3.03	2.27	2

Solving: Maximize throughput

Text Only inputs



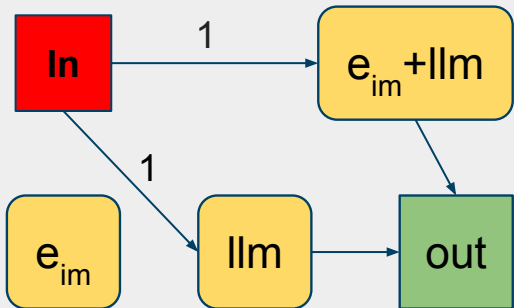
Text + Img inputs



	Img	text	Both	GPUs
e_{im}	10			1
llm		3.33		2
$e_{im} + llm$		3.03	2.27	2

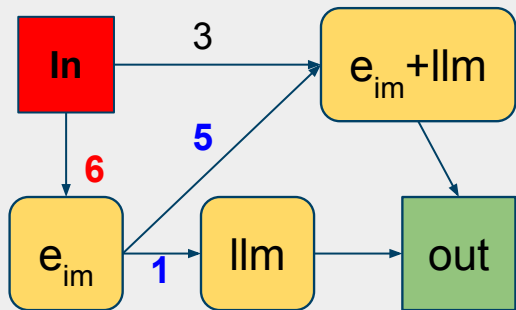
Solving: Maximize throughput

Text Only inputs



	Img	text	Both	GPUs
e_{im}	10			1
llm		3.33		2
$e_{im} + llm$		3.03	2.27	2

Text + Img inputs



	Work Demand	Replicas	GPUs
E_{im}	6 / 10	= 1	1*1=1
llm	(1+1) / 3.33	= 1	1*2=2
$E_{im} + llm$	(1+5) / 3.03 + 3 / 2.27	= 4	4*2=8



This process is NP Hard!

Key Trick: Cells

Cell (c): Power of 2 GPUs (1,2,4..): one complete model instance

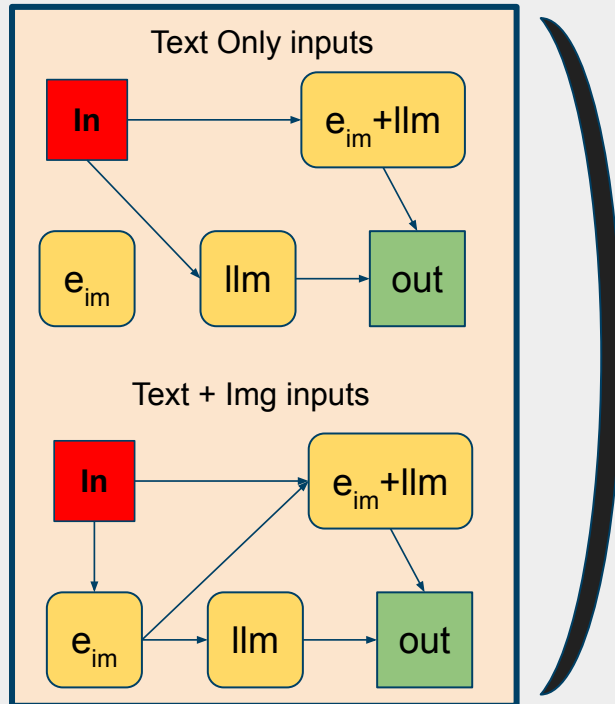
Internally: Planner solves the small optimization for that size -> throughput R_c

Externally: Cells are the **Lego Block** that we could compose to meet targets

Placement: Cells fit in **high-affinity** domains which hides the topology (executor graph)



Building efficient cells -> offline

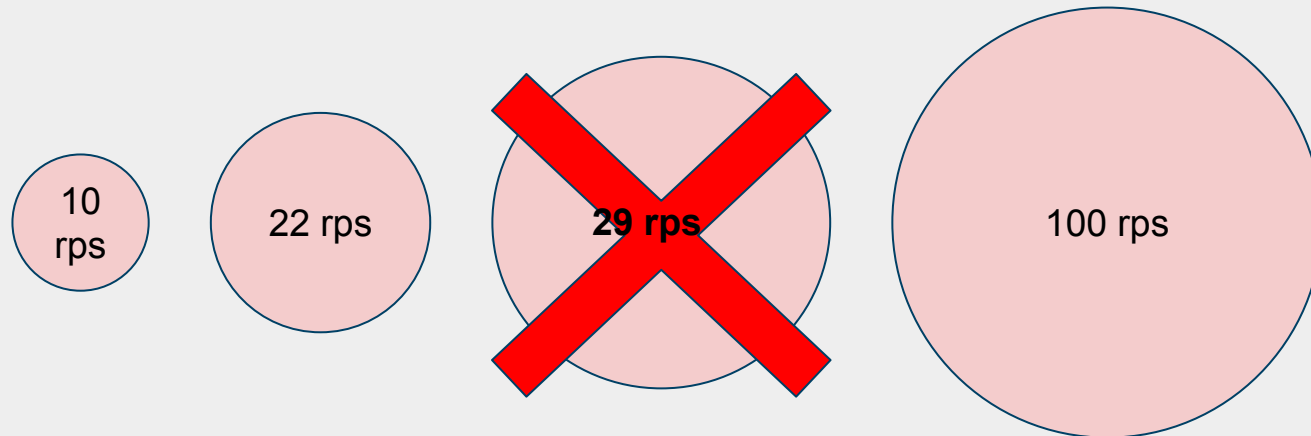


Cells of Size with G (#gpus) =
Powers of 2 (1,2,4,... MAX)

Building efficient cells

In Each Cell: Solve for maximum Throughput R_g -> obtain set of Cells

Filter out inefficient cells: If two smaller cells Throughput > larger cell, don't take cell



Online: Finding Optimal Cell mix

Objective:

1. **Maximize throughput given GPU budget N**
2. **Minimize GPUs given target throughput R**

For Maximizing throughput, we simply just Sum the R_c of these cells by dividing the budget into different cells in the offline phase.

For Minimizing GPUs for a target R , we greedily take the largest cell whose R_c doesn't exceed the remaining Gap until we reach target R



Distributed Runtime

```
from cornserve.task.base import Task
from cornserve.tasklib import LLMTask, EncoderTask

class MLLM(Task[Request, Response]):
    def post_init(self) -> None:
        # Initialize model components
        model_id = "Qwen/Qwen2.5-VL-7B-Instruct"
        self.llm = LLMTask(from_model_id=model_id)
        self.enc = EncoderTask(from_model_id=model_id)

    def invoke(self, request: Request) -> Response:
        # Encoder only invoked if there are images
        embeddings = []
        for image in request.images:
            embeddings.append(self.enc.invoke(image))

        return self.llm.invoke(
            text=request.text,
            image_embeddings=embeddings,
        )
```

Task Abstraction:

unit tasks (Encoder/LLM/Generator) +
composite task in Python

Record + Replay:

We first record the invoked subgraphs, then
we we replay it and invoke it

Evaluation -> Qwen 2.5 text/audio/image gen

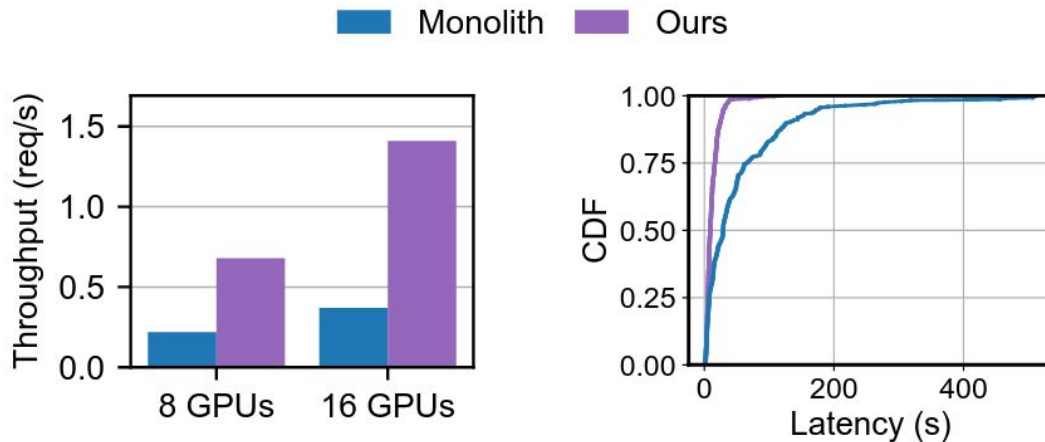


Figure 7: Qwen 2.5 Omni cell size and throughput

Figure 8: Qwen 2.5 Omni end-to-end request latency CDF.

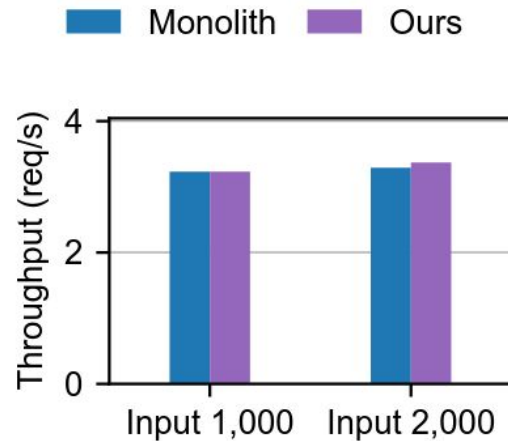
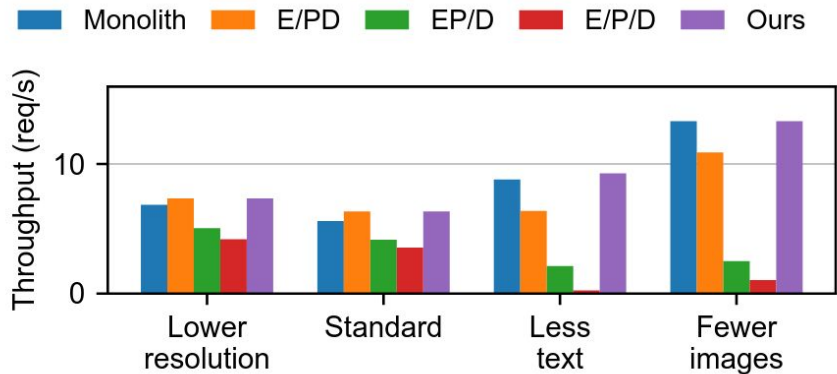


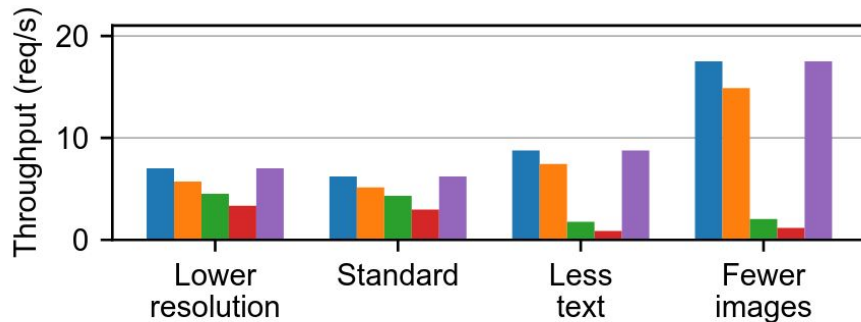
Figure 9: The throughput of serving Qwen-Image on 16 GPUs w. different input lengths.

Evaluation - Controlled Benchmarks



(a) *InternVL 3 38B*

InternVL-3-38B



(b) *Qwen 2.5 VL 32B*

Qwen2.5-VL-32B

Input	Output	Monolith			Ours		
		P50	P90	P99	P50	P90	P99
T + I + V + A	T	33.6	128.9	181.4	7.1	18.4	19.8
T + I + V	T	20.8	88.8	150.4	7.3	18.7	22.6
T + I + A	T	20.1	114.7	128.6	7.9	19.1	24.8
T + I	T	28.9	95.8	125.2	8.5	16.3	21.0
T + I + V + A	A	26.9	156.9	167.0	24.8	36.5	37.8
T + I + V	A	88.8	236.5	340.5	21.9	36.7	44.3
T + I + A	A	29.6	182.5	284.5	23.1	33.6	37.7
T + I	A	86.2	288.6	375.2	21.3	30.4	39.6

Table 2: Qwen 2.5 Omni latency statistics by request types on 16 GPUs. I stands for image, V for video, A for audio, and T for text.

Conclusion

- Cornserve is able to provide an efficient Any-to-Any serving system through the planner which automatically finds an optimal deployment plan for the model

Limitations

- How to communicate between data flows (instant communications)
- Caching different data for each components

Summary

	LoongServe	Cornserve
Task	Text-only	Any-to-Any
Disaggregation	Prefill/Decode	Encoder/Prefill/Decode/MMDecoder
Granularity	Iteration-level scaling up/down	Request-level resource planning
Scheduling	Dynamic allocating and batching	Offline profiling and static routing
Communication	Zero-overhead KV Cache transmission	Isolated caches; transmit only intermediates (hidden states/logits...)

Thank You!

Q & A